Project Proposal

An Adaptive Wave Director for a Maze Tower Defense

A contextual bandit that answers the weaknesses of the player's build, measured by whether players rebuild

[Student name]
ID: [student ID]

[Course code and name]
Advisor: [advisor name]
Committee: [committee members]

[Programme]
[Faculty / College]
[University]

October 2026

Table of Contents

1. Introduction 6
1.1 Background and Problem Statement 6
1.2 Goals and Objectives 6
1.3 Hypotheses 7
1.4 Scope 7
In scope 7
Out of scope 7
Limitations and requirements 8
1.5 Document Conventions 8
2. Literature Review 9
2.1 Adaptive Game AI 9
2.2 Related Works 9
2.3 Algorithms and Techniques 9
2.3.1 Flow-field pathfinding with diggable terrain 9
2.3.2 Deterministic simulation 10
2.3.3 Contextual bandits and Thompson sampling 11
2.3.4 Measuring restructuring 11
2.4 Approaches Considered and Rejected 12
2.5 Tools for Development 12
3. Deliverable Details 13
3.1 Project Overview 13
3.2 System Architecture 14
3.3 Simulation Core 15
3.4 Wave Planning Seam 17
3.5 Adaptive Director 17
3.5.1 Build profile 18
3.5.2 Strategies and threat budget 18
3.5.3 Layers and selection 19
3.5.4 Reward and credit 19
3.6 Telemetry, Replay and Analysis 19
3.7 Current Status 20
4. Evaluation Methodology 21
4.1 Gates 21
4.2 Offline Evaluation 21
4.3 Player Study 21
4.4 Threats to Validity 22
5. Plan of Action 23
5.1 Tasks 23
5.2 Schedule 24

5.3 Monitoring and Evaluation 25
5.4 Risks 25
6. References 26

List of Figures

Figure 1 The flow-field debug view on a generated 48 × 48 map
Figure 2 Session start
Figure 3 A build phase
Figure 4 The director at work (development version)
Figure 5 A wave in progress
Figure 6 Assemblies and dependencies
Figure 7 One simulation tick, and the two records a session leaves behind
Figure 8 The director pipeline
Figure 9 After a wave

List of Tables

Table 1 Hypotheses and how each is tested
Table 2 Comparison with related adaptive systems
Table 3 Rejected approaches and the reason
Table 4 Development tools
Table 5 Code base at the time of writing
Table 6 Hard invariants
Table 7 Build profile features (starting definitions; frozen before the pilot)
Table 8 What is built and what is planned
Table 9 Gates
Table 10 Work packages
Table 11 Semester 1 schedule
Table 12 Break and semester 2 schedule
Table 13 Input, process, output and benefit
Table 14 Main risks and mitigation

Abstract

Maze tower defense games ask the player to shape the enemies' route with walls and to cover it with towers. A competent player usually finds one strong layout within a few waves and then repeats it for the rest of the session, because the waves are scripted and do not react to what was built. This project proposes an **adaptive wave director**: a component that reads the player's current build at the start of every build phase and chooses the *shape* of the next attack (which enemy classes, which entry points, which weakness to test), while the total threat of each wave is held equal to that of a static baseline.

The director has three strictly ordered layers: hard constraints that veto illegal or unfair choices, an authored heuristic that scores each strategy from a five-feature build profile, and a learned correction bounded to ±1 that comes from a binned contextual bandit with Thompson sampling over discounted Beta posteriors. The learner is rewarded for *behaviour change* (how much the player restructured the build after the wave), not for defeating the player.

The game runs on a deterministic, headless simulation core written in plain C# (fixed 0.02 s ticks, seeded PCG32 streams, state hashing), with Unity 6 acting only as input and presentation. The same rules run under .NET for unit tests, replays and roughly a million offline episodes. Every session leaves a replay and one telemetry row per wave, from which the primary measure, the Shannon entropy of where the player builds, is computed after the fact. The hypothesis is tested offline against scripted players (gates G1–G4) and then in a within-subject study (n ≈ 24) that compares the director with the static baseline. The claim under test is not that the director wins more, but that players **rebuild more**.

1. Introduction

1.1 Background and Problem Statement

Tower defense is a strategy genre in which the player places defensive structures to stop waves of enemies from reaching a goal. In a *maze* tower defense the player also builds walls, so the route itself is part of the design: a longer route keeps enemies under fire for longer. The genre rewards planning, but it also has a well-known failure mode. Once the player discovers a layout that works, the scripted waves keep testing the same thing, and the best response is to repeat that layout. From that point the player stops making structural decisions, and the game turns into upkeep.

Games have long adjusted themselves to the player. Dynamic difficulty adjustment (DDA) scales enemy strength up or down from the player's performance [3], and director systems such as the one in *Left 4 Dead* shape pacing from an estimate of player stress [4]. These systems mostly optimise *difficulty* or *pacing*. They do not try to change *how* the player plays. A harder wave that the same layout still handles leaves the player's behaviour unchanged, and a director that simply turns difficulty up can make the comparison with a baseline meaningless, because "it was harder" explains any difference.

This project targets the behaviour directly. The director reads the weaknesses of the current build (an exposed chokepoint, an undefended outpost, a stack of towers in one place, reliance on one damage type, little anti-air) and sends a wave that exploits one of them, while spending exactly the same threat budget as the static wave it replaces. If the player answers by restructuring, the director has done its job; if the player ignores it and loses, it has not. The research question is therefore:

> **RQ.** Does a wave director that adapts the *shape* of each attack to the player's build, at constant difficulty, make players restructure their defences more often than a static escalation of the same strength?

Two engineering problems stand between the idea and a valid answer. First, a learning director has very little data: a session lasts about 25 waves, so the learner gets about 25 decisions per player. Second, the comparison must be fair: both conditions must see the same map, the same random shop offers and the same total threat at every wave number. Most of the technical work in this proposal exists to meet those two constraints.

1.2 Goals and Objectives

The project delivers a playable game, an adaptive director inside it, an offline evaluation, and a player study. Each objective below can be checked off against the finished product.

1. **A deterministic simulation core.** The same map, seed and input log give the same state on every tick, in the Unity editor (Mono) and headless (.NET). Verified by per-tick state hashes in replays (invariant I1).
2. **A playable maze tower defense** on that core: walls that are diggable terrain rather than obstacles, towers, an ore economy, several enemy movement classes and a static escalation baseline. The game design is specified in the separate GDD.
3. **A build profiler** that turns the board into a five-feature vector (`maze_length`, `breach_vulnerability`, `chokepoint_reliance`, `tower_concentration`, `damage_type_mix`).
4. **A strategy pool and a shared threat cost table** so that every strategy spends an identical threat budget at a given wave number (invariant I2).

5. **A three-layer director** (constraint veto, authored heuristic, bounded learned correction) with a binned Beta-posterior estimator, a kernel prior for thin cells, Thompson sampling and a discounted update.
6. **Telemetry and replay:** one JSON line per wave and a replay file per session, from which the primary metric is reproduced exactly.
7. **An offline harness** with scripted player policies that runs the evaluation ladder and gates G1–G3 before any participant plays.
8. **A within-subject player study** (pilot n = 5, main study n ≈ 24) that tests gates G5–G7.

1.3 Hypotheses

Table 1: Hypotheses and how each is tested

ID	Hypothesis	Test
H1	The learner identifies the best strategy for a context when one exists.	Offline, synthetic threshold environment: ≥ 85% best-strategy selection within 300 episodes (G1).
H2	A graded reward learns faster than a binary one.	Offline, 20 seeded runs, episodes to G1 accuracy (G3).
H3 (primary)	Players restructure more under the adaptive director than under the static baseline.	Within-subject, placement entropy per session, Wilcoxon signed-rank, $p < .05$ and $dz \geq 0.5$ (G5).
H4	The director is not perceived as less fair.	Non-inferiority: fairness rating no worse by more than 0.5 on a 7-point scale (G6).
H5	Players notice the adaptation.	≥ 70% say unprompted that the enemies adapted (G7).

1.4 Scope

In scope

- A single-player, mouse-and-keyboard maze tower defense for Windows PC, built in Unity 6 (URP).
- A procedurally generated 48 × 48 tile map from a seed, with rocks, ore deposits and one to three enemy entry points per wave.
- Walls and towers as diggable terrain: a path to the core always exists, and enemies decide by cost whether to walk round or dig through.
- Enemy movement classes that the strategies vary: ground walkers, sappers (cheap digging), flyers (straight line over everything), raiders (target ore collectors) and siege units (outrange towers).
- The director, the offline harness, telemetry, replay tools and the analysis scripts.

Out of scope

- Difficulty scaling as an objective. Difficulty is held constant on purpose; a change that makes the game harder without changing how the player builds counts as a regression.
- Deep reinforcement learning, policy networks and generated (LLM) behaviour: about 25 decisions per session cannot train them, and they would break determinism (Section 2.4).

- Multiplayer, mobile platforms, monetisation and final art. Art, sound and the look of the interface are polished only to the level the study needs.

Limitations and requirements

- The study build is Windows 64-bit with the Mono scripting backend; a build guard refuses any other target, because Mono is the runtime on which determinism was verified.
- Minimum hardware: a PC that runs Unity 6 URP at 1920 × 1080 and 60 frames per second (a quad-core CPU, 8 GB RAM, a DirectX 11 graphics card).
- The director decides once per wave boundary, so it adapts on the timescale of waves, not seconds.
- The thresholds that bin the context and the definition of the primary metric are frozen before the pilot; after the build freeze only crash fixes are allowed.

1.5 Document Conventions

Gameplay (screens, controls, tower and enemy rosters, economy) is specified in the separate Game Design Document and is summarised here only where the technical design depends on it. Code identifiers are written in `monospace`. Invariants of the code base are referred to as I1–I12 (Table 6) and evaluation gates as G1–G7 (Table 9). Costs in the flow field are integers scaled by 10, so a cost of 14 is 1.4 tiles.

2. Literature Review

2.1 Adaptive Game AI

Dynamic difficulty adjustment. Hunicke argued for adjusting a game's difficulty to the player at run time and for doing it without the player noticing the hand on the scale [3]. Most DDA work since then estimates skill from performance (health lost, time taken, win rate) and moves a difficulty parameter toward a target. Sutoyo et al. applied DDA to tower defense, adjusting the waves to the player's performance [5]. The present project borrows the per-wave decision point but rejects the objective: it keeps difficulty fixed and changes the composition.

AI directors. The *Left 4 Dead* director monitors an estimate of each player's emotional intensity and builds up, sustains and relaxes the population of enemies to produce dramatic pacing [4]. It is rule-based and does not learn from the individual player. Its separation of "what to send" from "how the enemies behave" is the same seam this project uses (Section 3.5).

Dynamic Scripting. Spronck et al. adapt opponent behaviour online by keeping a weight for each rule in a rule base and adjusting the weights after each encounter according to the outcome [6]. It is the nearest neighbour of this project and is discussed first. The differences are: a Bayesian posterior per context cell instead of heuristic weights; an explicit per-player build context instead of one global weight vector; and an objective of build diversity rather than difficulty balance.

2.2 Related Works

Table 2: Comparison with related adaptive systems

System	What it adapts	Learns per player	Objective	Holds difficulty constant
Hunicke, DDA [3]	Supply and enemy strength	No (rules)	Keep the player in a target difficulty	No: difficulty is the variable
Left 4 Dead director [4]	Enemy population and pacing	No (rules)	Dramatic pacing	No
Dynamic Scripting [6]	Rule weights of opponent scripts	Yes, one global weight vector	Win rate / balance	Partly
Sutoyo et al., DDA in TD [5]	Wave difficulty	No	Match difficulty to performance	No
This project	Wave *shape*: classes, entries, targets	Yes, posterior per (context bin, strategy)	Player restructures the build	Yes: equal threat budget (I2)

Two further sources shape the method rather than the system. Al Enezi and Verbrugge (AIIDE 2023) is the closest methodological model for the player study [7]. The calibration check on the learner's posteriors (Brier score and expected calibration error) follows Brier [17] and Guo et al. [18].

2.3 Algorithms and Techniques

2.3.1 Flow-field pathfinding with diggable terrain

Many tower defense games give each enemy its own path from an A-star search. When hundreds of agents share one goal it is cheaper to compute one *flow field*: a single shortest-path tree grown

outward from the goal, in which every tile stores its cost to the goal and the neighbour to step to next [8]. Every agent then follows the arrows in constant time per step.

This project builds the field with a weighted flood fill in the style of Bellman–Ford with a work queue (SPFA) [9], over the whole map, synchronously. Moving from a tile to a neighbour costs 10 (orthogonal) or 14 (diagonal), multiplied by the terrain cost of the tile being left. The unusual rule is that walls and towers are **not obstacles**. They are expensive, destructible terrain (a wall's dig cost defaults to 15, a tower's to 20), and an agent standing on one chews through it. Because every tile is passable at some cost, a path to the core always exists, and agents trade "walk round" against "dig through" without any scripted choice. A corner-cut rule stops agents slipping diagonally between two solid tiles.

$$cost(n) = \min_{\text{neighbours } m} \left[cost(m) + step(n, m) \times terrain(n) \right], \quad step \in \{10, 14\}$$

Several fields are built by the same flood: the ground field; a sapper field in which player-built tiles cost a factor of 0.2 (mountains keep full price, so sappers do not tunnel the same rock every time); and a raid field that floods out from the player's ore collectors. Flyers ignore the fields and fly straight. On this grid a full rebuild of both main fields takes about 0.3–0.6 ms under .NET and 0.8–1.1 ms in the Unity editor (measured on a 38 × 38 board with up to a quarter of it built), which is why chunked, threaded and incremental variants were removed rather than kept.

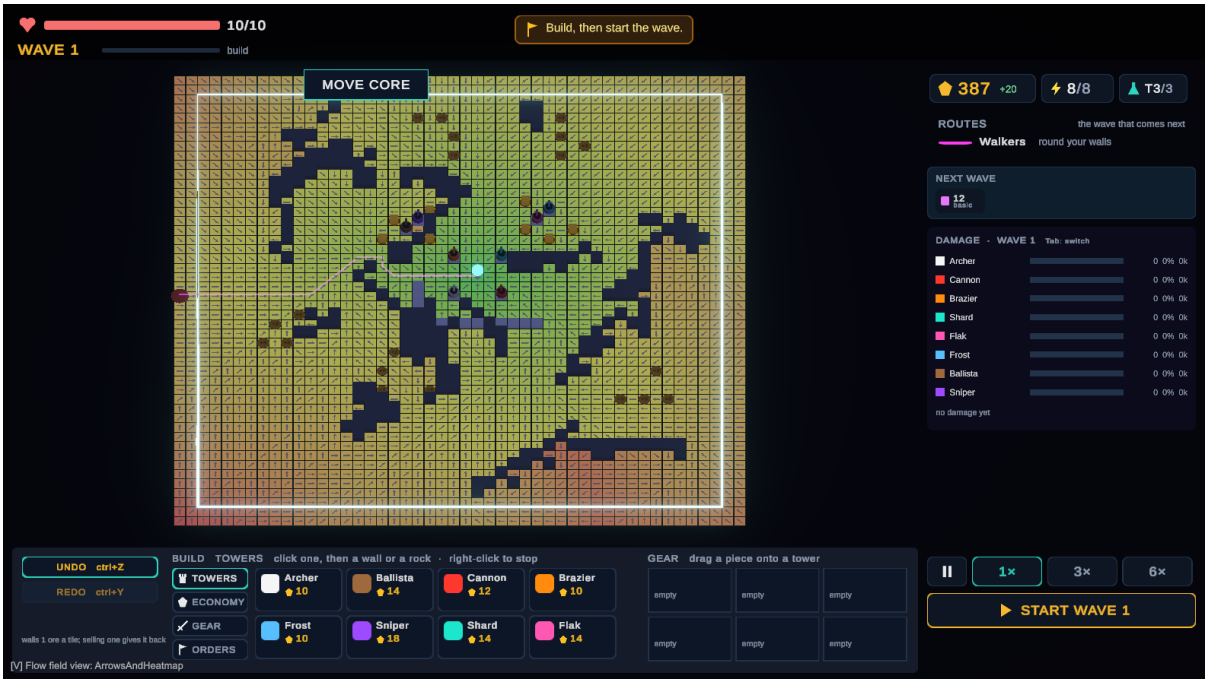


Figure 1: The flow-field debug view on a generated 48 × 48 map: the heat map is the cost to the core (green near, red far); dark tiles are rocks; the pink line is the route walkers will take.

2.3.2 Deterministic simulation

A learning director must be evaluated on replays and on about 10^6 offline episodes, and the study must show that both conditions saw comparable waves. Both require that the same inputs give the same game. Unity's frame loop does not: results depend on frame rate and on the speed setting, the update order of objects is undefined, and coroutine timing snaps to frames. The game rules were therefore moved out of `MonoBehaviour`s into a plain C# simulation that advances in fixed ticks of 0.02 s. The techniques used are standard but must all hold at once:

- **Seeded random streams.** PCG32 [10], one named stream per purpose (map, shop, waves, director), so a new use of randomness never shifts an existing one.

- **Bit-exact maths.** No `Math.Log`, `Math.Exp`, `Math.Pow` or trigonometry (these differ in the last bit between runtimes); a project `DetMath` provides deterministic versions. Chained float expressions are cast to `float` step by step so that the JIT cannot keep extra precision.
- **State hashing.** A 64-bit FNV-1a hash [11] of the whole state, floats by bit pattern, after every tick. A replay stores the seed, every command with its tick, and the hashes.
- **Whole-tick combat.** A tower's hit resolves instantly or after a delay fixed at the moment of firing; projectile flight exists only in the view (invariant I12).

2.3.3 Contextual bandits and Thompson sampling

Choosing a strategy each wave, observing a reward and choosing again is a *contextual bandit*: there are several arms (strategies), a context (the build profile) and a reward that depends on both. Thompson sampling [12] selects an arm by drawing a plausible value from each arm's posterior and taking the largest; it explores in proportion to uncertainty and has no exploration rate to tune [13, 14]. Linear contextual bandits such as LinUCB [15] assume the expected reward is linear in the context. That assumption is rejected here: in an inspected open-source implementation the best arm depended on a *threshold* in one feature, which a linear model cannot represent, and this domain has the same structure (a maze is either long enough to make sappers pay, or it is not).

The estimator is therefore **binned**: one context feature is discretised into 5 bins, and each (bin, strategy) cell holds a Beta(α, β) posterior, 5 × 4 = 20 cells. Beta variates are drawn as the ratio of two Gamma variates using Marsaglia and Tsang's method [16]. Because the update below never lets a shape parameter fall below 1, the small-shape branch of that method is not needed. Thin cells borrow strength from a Nadaraya–Watson kernel estimate [19, 20] over a 128-entry ring buffer of recent (profile, strategy, reward) triples, Gaussian-weighted (σ = 0.15) in a two-dimensional subspace of the profile:

$$\hat{Q}(x, a) = \Sigma_i K(x, x_i)\, r_i \,/\, \Sigma_i K(x, x_i), \quad \alpha_0 = 1 + k\cdot\hat{Q}, \;\; \beta_0 = 1 + k\cdot(1 - \hat{Q})$$

The player learns while the director learns, so the environment is non-stationary in both directions. Rewards are therefore discounted, in the spirit of discounted bandits for switching environments [21], but decayed toward the uniform prior Beta(1, 1) rather than toward zero, which keeps α, β ≥ 1. Fractional rewards are made conjugate by a Bernoulli draw:

$$\tilde{r} \sim Bernoulli(r), \;\; \alpha \leftarrow 1 + G(\alpha - 1) + \tilde{r}, \;\; \beta \leftarrow 1 + G(\beta - 1) + (1 - \tilde{r}), \;\; G = 0.95$$

2.3.4 Measuring restructuring

The primary measure is the normalised Shannon entropy [22] of where the player builds. Every tile the player builds after wave 1 (wall tiles and tower tiles, kept even if later sold or destroyed) is counted in its region of a fixed 6 × 6 partition of the map:

$$H = -\Sigma_r\, p_r \ln p_r \,/\, \ln K$$

where p_r is the share of counted tiles in region r and K the number of regions that contain a buildable tile. H = 0 means everything went into one region (repairing the same spot); H = 1 means building was spread evenly. It is computed offline from telemetry, so its definition can change at no cost until it is frozen before the pilot. Its known weakness, that a player who slowly extends one maze over the whole map also scores high, is cross-checked with the per-wave `restructure` term and the number of tiles and towers sold.

2.4 Approaches Considered and Rejected

Table 3: Rejected approaches and the reason

Approach	Why it was rejected
Linear contextual bandit (LinUCB-style)	Cannot represent the thresholds this domain is built around; documented failure in an inspected implementation.
Deep RL / policy network	About 25 decisions per session cannot train one.
ε-greedy exploration	Thompson sampling is better motivated and has no ε to tune.
LLM-generated behaviour, hosted decision models	Break determinism (no seed control, remote versioning), cost 70–500 ms per call, and do not learn from the individual player.
Walls as impassable tiles	Needs path validation and revert logic; diggable terrain guarantees a path and removed a whole class of bugs.
Chunked or threaded flow field	A synchronous rebuild costs about a millisecond here; the complexity bought nothing measurable.

2.5 Tools for Development

Table 4: Development tools

Tool	Use
Unity 6 (6000.4.10f1), URP	Host application: input, rendering, interface (uGUI and TextMeshPro), the study build.
C# / .NET 9	The simulation, the director and the harness, compiled both inside Unity (Mono) and headless (CoreCLR).
NUnit and Unity Test Framework	One test suite (661 tests at the time of writing) run by both Unity and `dotnet test`.
Thesis CLI (project tool)	Headless `run`, `replay`, `pin`, `telemetry`, `entropy`, `balance` and `bench` commands.
Git and GitHub	Version control; every result file records the commit and the exact command that reproduces it.
Python	Analysis of telemetry and the study statistics.
Graphviz	Architecture and pipeline diagrams.

3. Deliverable Details

3.1 Project Overview

The game is a single-player maze tower defense seen from above. The player places a core on a generated map, builds ore collectors on deposits to earn income, draws walls to lengthen the enemies' route, and places towers on walls or rocks to shoot them. Enemies enter from one to three edges each wave and follow the flow field to the core; a wave is lost when enough of them reach it. The full design is in the GDD. Figures 2–5 are captures of the current prototype.

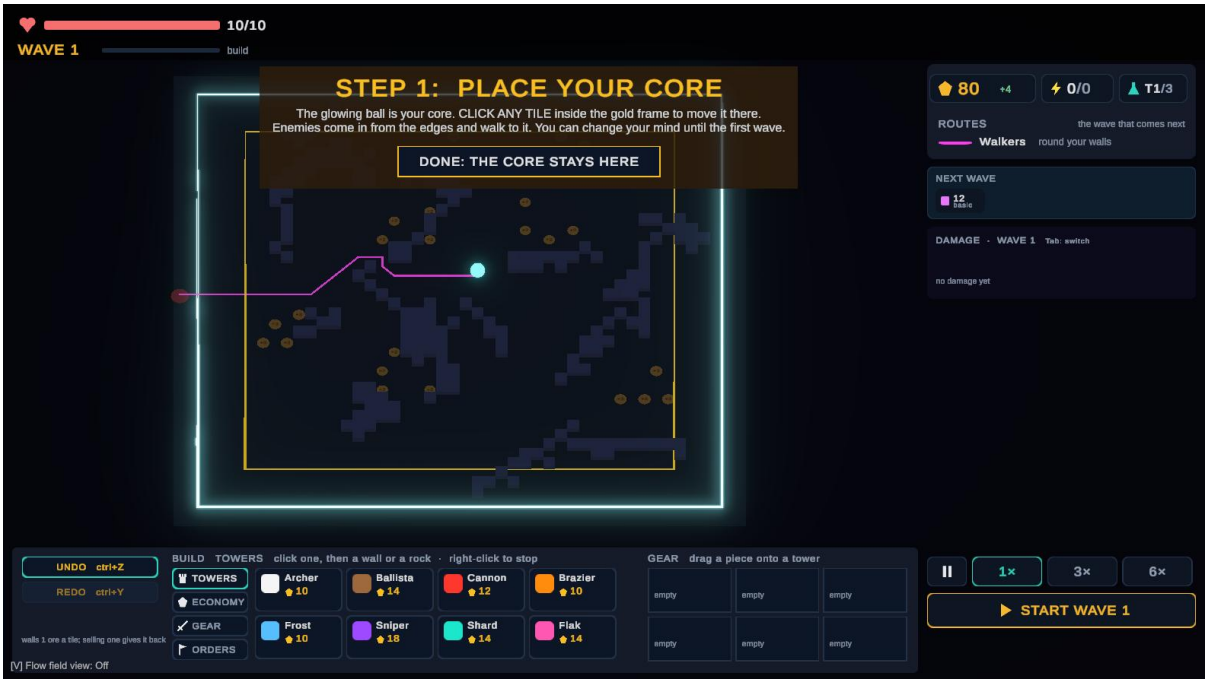


Figure 2: Session start: the player chooses where the core stands inside the gold frame. Brown discs are ore deposits, dark tiles are rocks; the pink line is the route of the first wave.

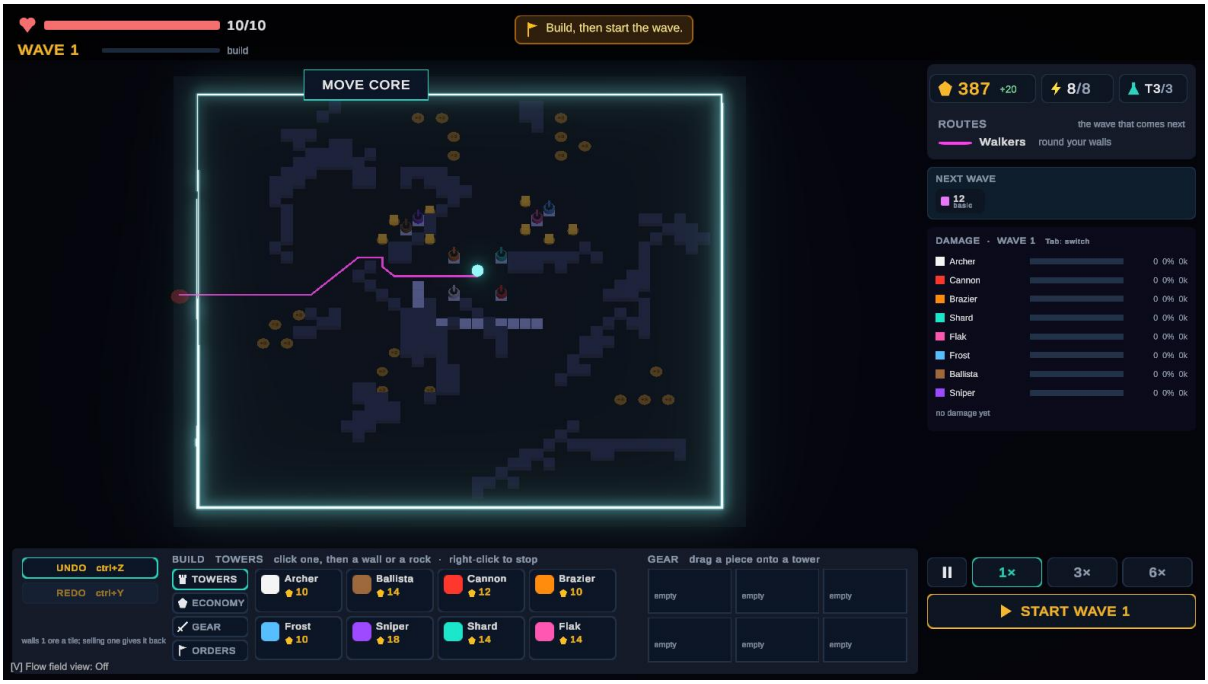


Figure 3: A build phase. Towers stand on walls round the core and the collectors; the right panel shows ore, powered towers and research tier, the planned next wave and the damage per tower of the last wave.

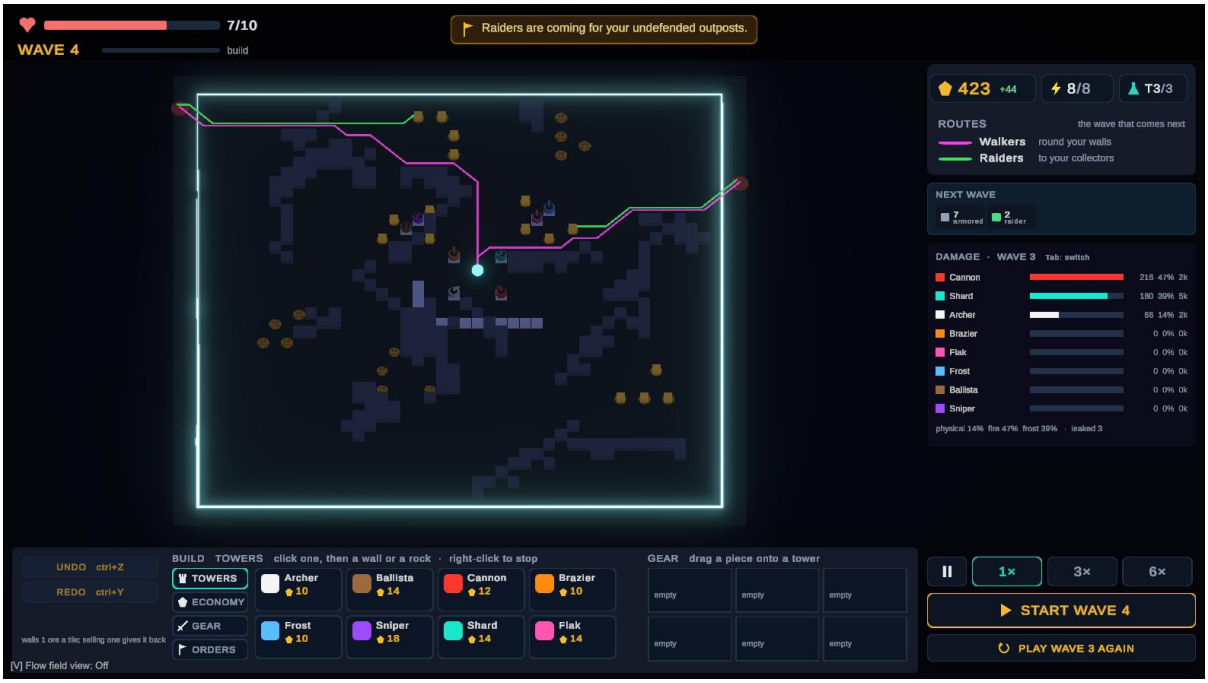


Figure 4: The director at work (development version). The player left collectors unguarded, so the next wave is planned as a raid: the banner announces it, the chips show 7 armoured enemies and 2 raiders, and a second route line (green) shows where the raiders will go.

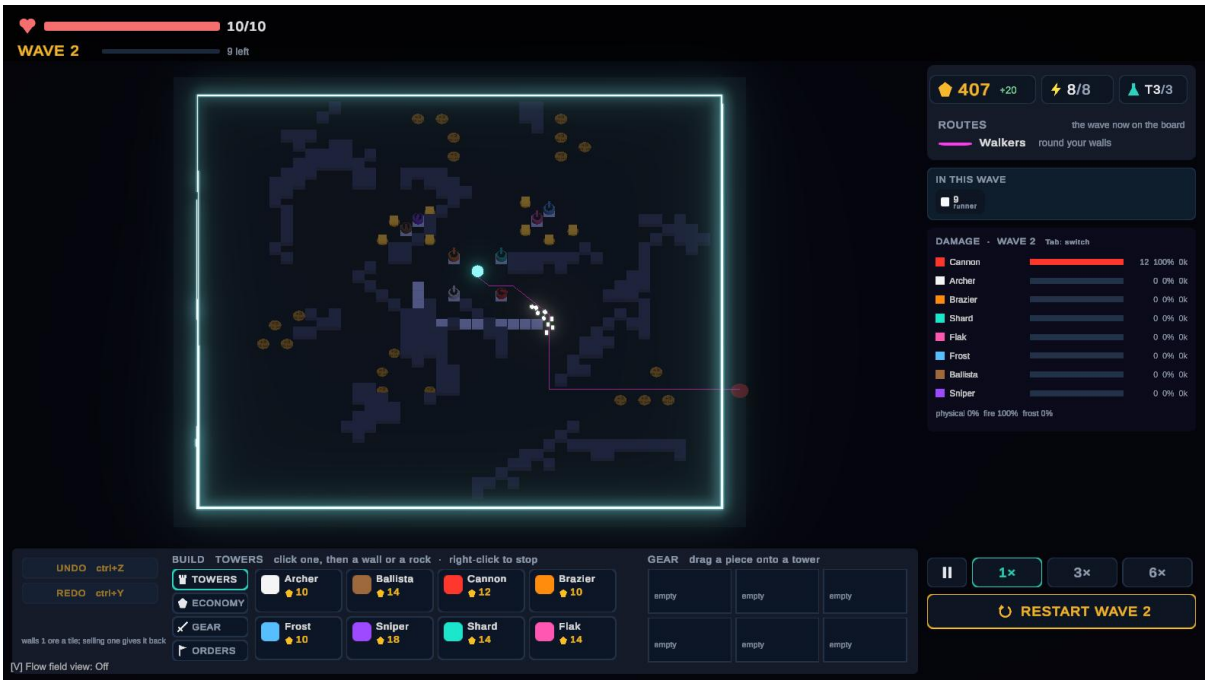


Figure 5: A wave in progress: runners follow the route past the wall line while the cannon takes them under fire; the damage panel fills during the wave.

3.2 System Architecture

The code is split into a pure C# core and a thin Unity host (Figure 6). Every game rule lives in `Assets/Thesis`, which may not reference `UnityEngine` (invariant I3, enforced by assembly definitions and by a test). The Unity scripts only translate input into commands, run the simulation in fixed ticks, and draw its state. The same assemblies compile headless under .NET, which is how the tests, the replay tools and the offline harness run.

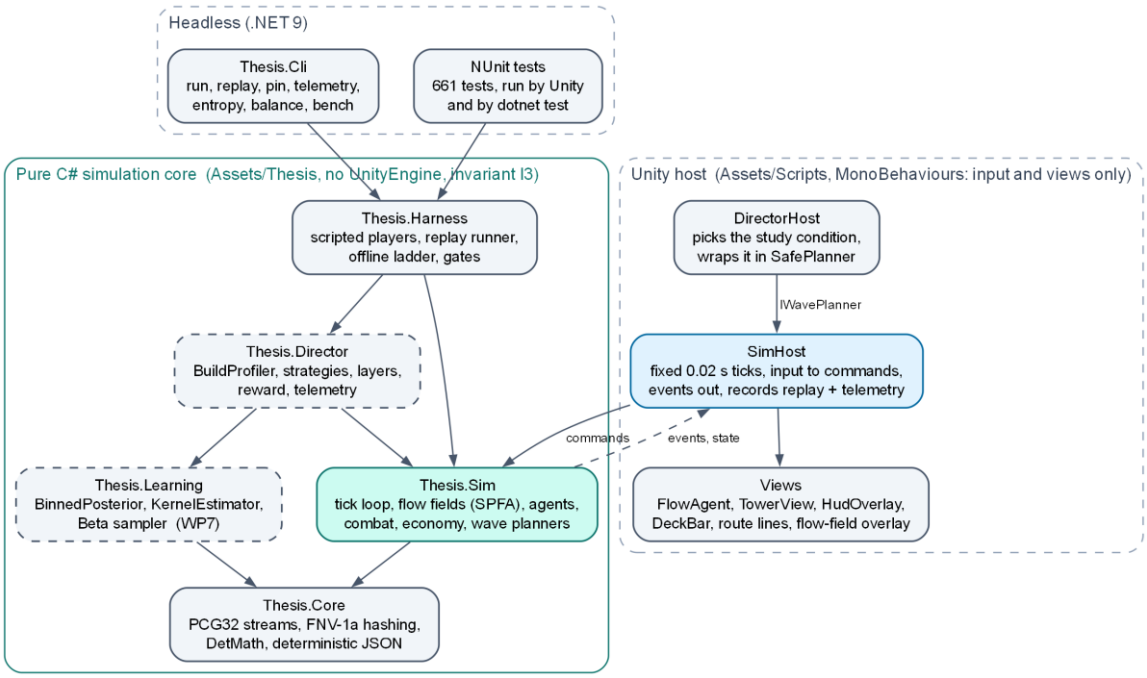


Figure 6: Assemblies and dependencies. Dashed boxes are designed but not yet built (Thesis.Learning, Thesis.Director).

Table 5: Code base at the time of writing

Part	Lines of C#	Notes
Simulation core (`Assets/Thesis`)	≈ 13,700	Core, Sim, Director (telemetry), Harness
Unity host and views (`Assets/Scripts`)	≈ 10,000	SimHost, views, HUD, editor tools
Tests (`Assets/Tests`)	≈ 15,100	661 tests, both runners

3.3 Simulation Core

`Simulation.Tick()` advances the whole game by one fixed step in a normative order that the tests assert (Figure 7). Player input never touches state directly: a click becomes a `SimCommand` stamped with the tick at which it applies, and placement legality is decided by one function, `Placement.CanPlace`, which the ghost preview also calls, so the preview can never disagree with a real click (I9). The Unity host drains an accumulator of scaled frame time into ticks (at most 40 a frame), so the game speed buttons change how many ticks run per frame, never what a tick does.

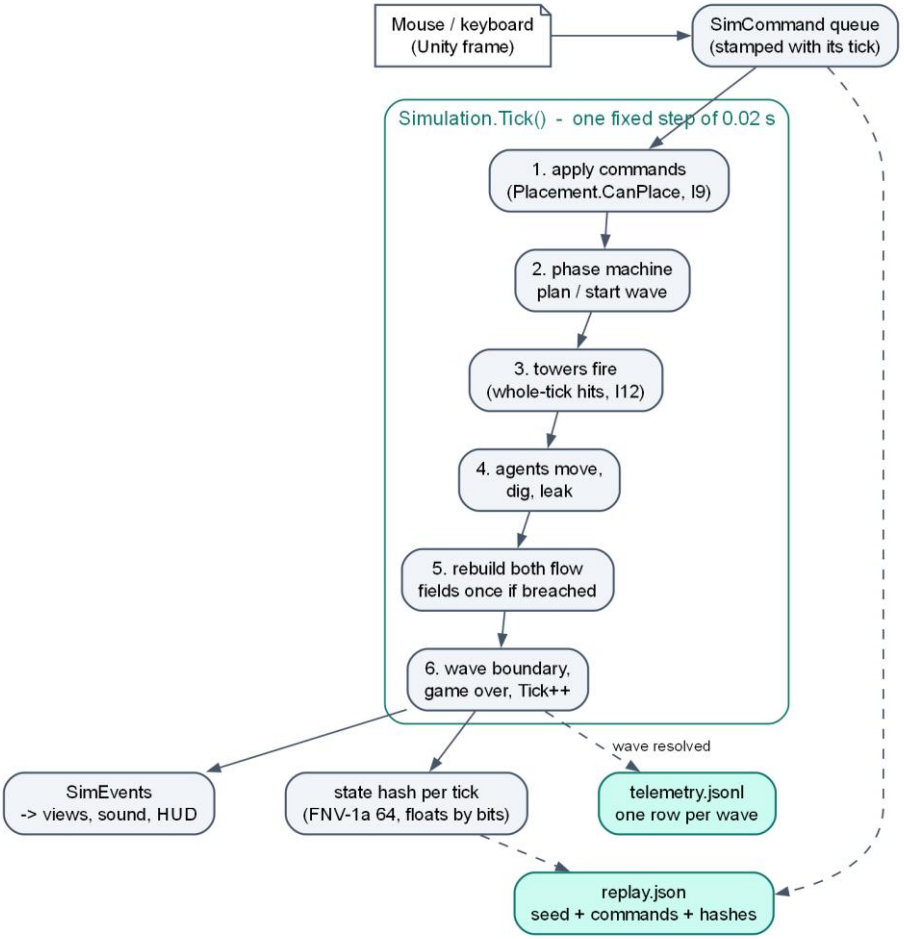


Figure 7: One simulation tick, and the two records a session leaves behind.

The rules the rest of the design depends on are written down as invariants and enforced by tests (Table 6).

Table 6: Hard invariants

#	Invariant	Why
I1	Same map, seed and input log give an identical outcome; fixed timestep; seeded PCG only.	Replays, regression tests, comparable conditions.
I2	Every strategy spends an identical threat budget at a given wave number.	Otherwise "the adaptive condition was harder" explains the result.
I3	The estimator and the rules have no Unity dependency.	Headless runs at ≈ 10^6 episodes; unit tests.
I4	The learned term is an additive correction bounded to ±1 on an authored score.	Cold-start safety; the ablation comes for free.
I5	Hard constraints veto with $-\infty$ before the learner is consulted.	Legality and fairness above learning.
I6, I7	Thresholds frozen before the pilot; no code changes after the build freeze.	Participants play the same game.
I8	No director logic on the per-frame path; ≤ 16 ms per wave boundary.	No visible cost (G2).
I9	`Placement.CanPlace` is the single source of placement legality.	Preview, click and harness agree.

#	Invariant	Why
I10	Shop offers are a pure function of (seed, wave, reroll count).	Both conditions see the same offers.
I11	The director shapes waves only; never the shop, prices, tower stats or budget.	The player's power is the same in both conditions.
I12	Tower attacks resolve in whole ticks; projectiles are view-only.	Frame rate never enters the rules.

Determinism was verified on 2 October 2026 by recording sessions in the Unity editor and headless and replaying each on the other runtime: a 25-wave, 111,861-tick episode with 466 commands matched on every tick in both directions, as did sessions recorded through the real Unity frame loop at speeds from x1 to a 20-tick-per-frame cap, paused and quit mid-wave. Ten pinned replays are re-run by every test run on the runtime that did not record them, so any change that alters the rules is detected immediately.

3.4 Wave Planning Seam

Everything that decides a wave sits behind one interface, so the static baseline, the development director and the final director are interchangeable study conditions:

```
public interface IWavePlanner {
    string Name { get; }                          // written to telemetry as the condition
    WavePlan PlanWave(WaveContext context);       // read-only: the state hash is checked
after the call
    void OnWaveResolved(WaveOutcome outcome);     // receives a copy
}
```

The next wave is planned as the build phase begins, so the player sees it (the enemy chips and the announcement) and can respond before pressing Start. The simulation hashes the whole state before and after `PlanWave` and throws if the planner changed anything; `ValidatePlan` checks the plan; and every planner runs inside a `SafePlanner` that falls back to the static baseline and logs an error if the planner throws. A planner failure therefore never corrupts a session. The static baseline (`EscalationPlanner`) grows enemy count, speed and hit points every wave and is the control condition.

A development version of the authored layer (`RoughDirector`) is already in the game (Figure 4). It reads the board (tower clusters, bare collectors, the dominant damage type, anti-air share, maze stretch) and scores raids, siege units, resistant enemies, flyers and sappers against a baseline score, with a repeat penalty. It is a prototype of layer 2 used for play-testing: it has no learned term and no threat cost table, and is not a study condition.

3.5 Adaptive Director

The director runs once per wave boundary (Figure 8). It records every intermediate value in a trace that is written into that wave's telemetry row, so the question "why did it choose X at wave 7?" is answered by one line of JSON.

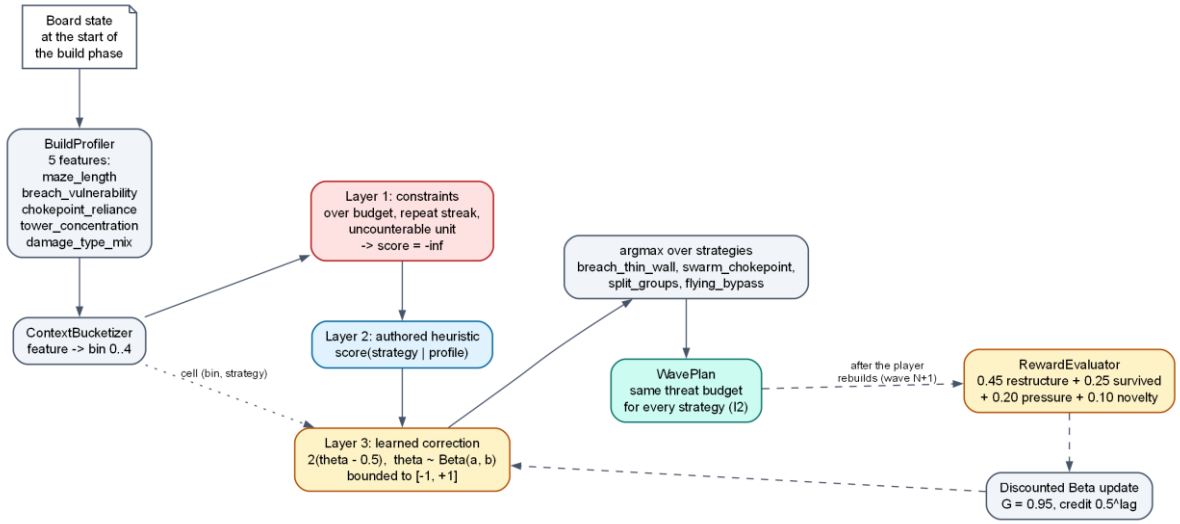


Figure 8: The director pipeline. The reward for wave N is known only when wave N+1 is planned, after the player has rebuilt.

3.5.1 Build profile

Table 7: Build profile features (starting definitions; frozen before the pilot)

Feature	Definition
`maze_length`	Cost of the ground route from the entry to the core, divided by the same cost on the empty map.
`breach_vulnerability`	Share of the route's cost that runs through diggable tiles: how much the route depends on walls holding.
`chokepoint_reliance`	1 − normalised entropy of tower damage over the route's tiles: damage concentrated on a few tiles.
`tower_concentration`	1 − mean tower distance from the towers' centroid ÷ map diagonal.
`damage_type_mix`	Normalised entropy of damage dealt per damage type (physical, fire, frost) last wave.

The features describe what the player *chose* to build. The random shop is a confound for the primary metric, because random offers push players to rebuild in both conditions; offers are therefore a pure function of the seed (I10), both conditions use the same seeds, and telemetry records what was offered, bought and sold.

3.5.2 Strategies and threat budget

Each strategy turns a threat budget and the wave context into a wave plan. The planned pool is `breach_thin_wall` (sappers at the thinnest wall), `swarm_chokepoint` (many weak enemies through the most relied-on chokepoint), `split_groups` (several entry points at once) and `flying_bypass` (flyers over the maze). The budget of wave w is the price of the static baseline's wave w in a shared cost table, and validation rejects any plan that misses it:

$$Budget(w) = Price(EscalationPlanner.Plan(w)), \quad |ThreatSpent(plan) - Budget(w)| \leq \varepsilon$$

Only the cost table is tuned, never per-strategy multipliers, so I2 cannot drift.

3.5.3 Layers and selection

- **Layer 1, constraints:** over budget, a strategy repeated too often, a unit the player cannot counter (flyers when no tower can hit them), or a plan whose composition lies within a fixed L2 distance of the previous wave's. A veto returns $-\infty$ and the learner is not called (I5).
- **Layer 2, authored heuristic:** a score per strategy from the profile. It is a complete director on its own and is the ablation condition.
- **Layer 3, learned correction:** for each surviving strategy, $\theta \sim \text{Beta}(\alpha, \beta)$ of its (bin, strategy) cell and a correction $2(\theta - 0.5) \in [-1, +1]$, or 0 while the cell is cold. The correction re-ranks close calls; it never selects on its own (I4).

```
foreach (var a in survivingStrategies) {
    float theta = SampleBeta(alpha[bin, a], beta[bin, a]);
    float corr  = IsCold(a) ? 0f : 2f * (theta - 0.5f);   // bounded [-1, +1]
    score[a]    = Heuristic(a, profile) + corr;
}
Play(ArgMax(score));                                      // ties: lowest strategy id
```

Every chosen strategy is announced before the wave with one authored line keyed by (bin, strategy). The lines are templated, never generated; they are part of the game feel and also the manipulation check for G7.

3.5.4 Reward and credit

$$r = 0.45 \cdot restructure + 0.25 \cdot survived + 0.20 \cdot pressure + 0.10 \cdot novelty$$

`restructure` is the change in the build profile after the wave (normalised L1, capped), `survived` is whether the core still stands (it stops a strategy profiting by ending the run: a defeated player builds nothing more), `pressure` is the share of the wave that came close to the core, and `novelty` is the distance from the previous wave's composition. Because a player may react to the last two or three waves, credit for an observed restructure is spread backward with weight 0.5^lag. A binary reward, $(\text{restructure} \geq \tau) \wedge \text{survived}$, is also implemented as the control for gate G3. The posteriors can be exported and imported, which allows a director trained against one player to be tested cold on another (the transfer condition).

3.6 Telemetry, Replay and Analysis

Every session writes two files. `replay.json` holds the seed, the configuration, every command with its tick and the state hashes; it is the exact record. `telemetry.jsonl` holds one row per wave: the plan that was sent, the outcome (spawned, killed, leaked, damage by type and by class, core hit points), the player's placements, sales and purchases around it, and, once the director exists, the profile, bin, scores, vetoes, reward components and the director's time. A row contains nothing the simulation does not determine (no wall-clock time) and numbers are formatted identically on both runtimes, so `thesis telemetry <replay>` regenerates the file byte for byte. The schema carries a version number and rows are append-only.

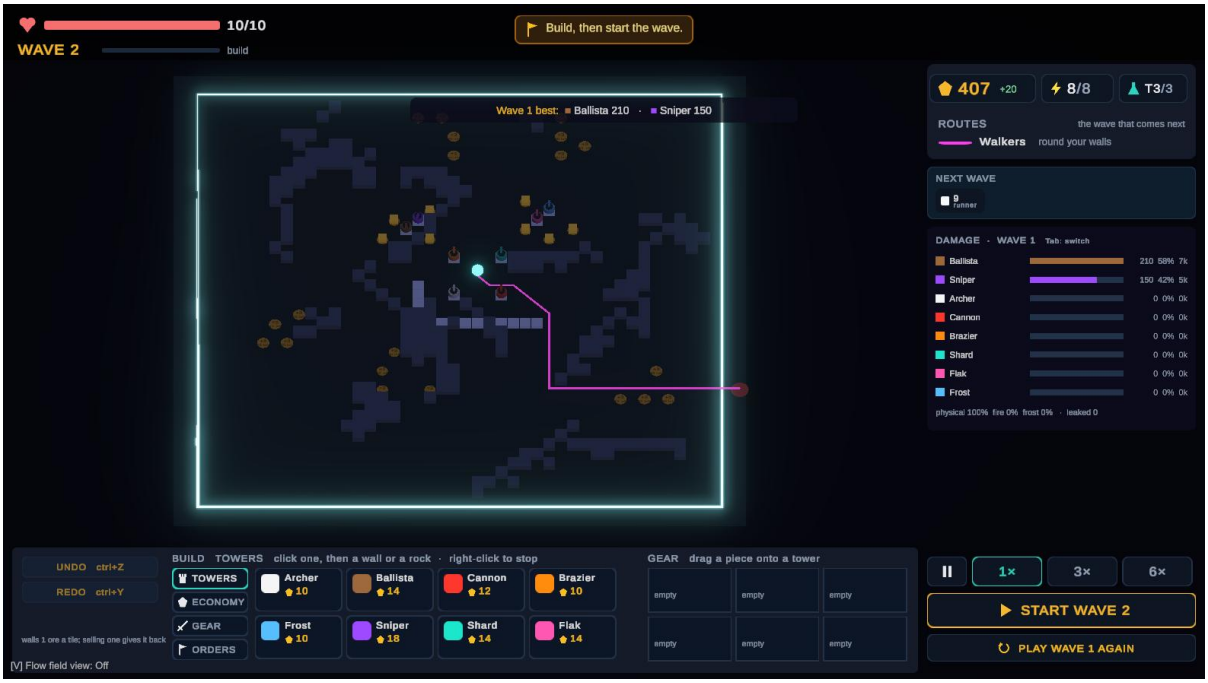


Figure 9: After a wave: the damage panel lists each tower's share of the damage, and the next wave is already planned from a different entry. The same numbers go into the telemetry row.

Telemetry already serves balancing. In a 77-wave tester session on 28 October 2026 it showed that the core was never under pressure for 66 waves, that the economy ran away (28 collectors, up to 877 ore banked), that enemy hit points compounding at 8% a wave created a late cliff, and that the development director stopped reacting after wave 11. Each finding became a change to the rules, and the next session is the check.

3.7 Current Status

Table 8: What is built and what is planned

Component	Status	Evidence
Deterministic core, replay, CLI	Done	Determinism result 2026-10-02; pinned replays in both runners
Combat, movement classes, economy	Done	8 towers and a pylon, 12 enemy types, ore deposits, research
Telemetry and placement entropy	Done	`thesis entropy`; schema versioned
Unity host, HUD, route lines	Done, iterating	Tester sessions on 2026-10-12, -14, -20, -28
Development director (layer 2 prototype)	Done	`RoughDirector`, Figure 4
Learning assembly (WP7)	Planned	Synthetic threshold environment as the known-answer test
Build profile, strategies, cost table (WP8–WP9)	Planned	Need the frozen feature formulas
Layers, full director, harness, gates (WP10–WP13)	Planned	

4. Evaluation Methodology

4.1 Gates

Each gate has a pass number and a fail number fixed before implementation, and the fail branch names the action, so a failure produces a decision instead of a moved goalpost.

Table 9: Gates

Gate	Pass	Fail → action
G1 learner works	≥ 85% best-strategy selection within 300 episodes, and beats uniform-random and round-robin	< 70% or no separation → implementation defect, or the context carries no information: revisit the features
G2 no visible cost	Director work ≤ 16 ms per wave boundary	Frame spike > 33 ms → make it incremental
G3 graded beats binary	Reaches G1 accuracy in fewer episodes than the binary control, 20 seeded runs	No difference → ship binary, report it
G4 enough data per bin	Pilot n = 5: median observations per visited bin ≥ 8 per session	Median < 4 → drop to 3 bins, document
G5 players rebuild more (primary)	n ≈ 24 within-subject, Wilcoxon $p < .05$, $dz \geq 0.5$	$p > .05$ or $dz < 0.2$ → report the negative result
G6 not unfair	Fairness no worse by > 0.5 on a 7-point scale (non-inferiority)	Worse → reward or announcement wording at fault
G7 players notice	≥ 70% identify unprompted that enemies adapted	< 50% → G5 uninterpretable until fixed

G1 passing does not license claiming G5: a learner can converge cleanly offline and change nothing about how a human plays. That outcome would itself be the finding.

4.2 Offline Evaluation

Before any participant plays, the harness runs the director against scripted player policies (idle, greedy detour builder, sentry, seller and sequences of these) in headless episodes. The expected ordering of the *offline ladder* is random < static escalation < authored heuristic < full director, measured on the reward and on the placement entropy the policies produce when they respond to waves. If the full director does not beat the authored heuristic, that must surface in semester 1, not in the study. G1 and G3 use a synthetic environment whose best arm flips at a threshold in the context, so the correct answer is known; it runs 10^6 episodes in seconds. A calibration report (Brier score and 10-bin ECE of the posterior means against realised rewards) is published next to G1.

4.3 Player Study

Within-subject design: each participant plays two sessions, one with the static baseline and one with the adaptive director. Condition order and map seed are counterbalanced with two seed sets, so no participant meets the same seed twice and each condition gets each seed and each position equally often. Each session is cut to the same number of completed waves for the comparison. Measures: placement entropy (primary), the restructure term, tiles and towers sold per wave, and secondary partitions (4 × 4, 8 × 8, walls only, towers only), all fixed before the study; a questionnaire for fairness (G6) and an unprompted question on whether the enemies adapted

(G7). Analysis: Wilcoxon signed-rank test on the per-participant pairs [23] with the effect size dz [24].

4.4 Threats to Validity

- **Difficulty confound:** controlled by the equal threat budget (I2) and by the director never touching the player's power (I11).
- **Shop luck:** offers depend only on (seed, wave, reroll count) and seeds are shared across conditions (I10).
- **Learning and order effects:** counterbalanced order; the within-subject design removes individual build style.
- **Metric validity:** region entropy measures *where* building happens, not directly how much the structure changes; cross-checked with the restructure term and sale counts.
- **Post-hoc tuning:** bin thresholds and the metric definition are frozen before the pilot (I6); the build is frozen before the study (I7).

5. Plan of Action

5.1 Tasks

Table 10: Work packages

Package	Content	Status
WP0–WP5	Port the game into a deterministic C# core; Unity host; replay and CLI; determinism proof	Done (2026-10-02)
WP-H	Hardening after the review: planner safety, DetMath, pinned replays	Done (2026-10-02)
WP-C1–C5	Combat core, movement classes, shop and telemetry, tower UI, roster and balance check	Done (2026-10-12)
Gameplay rounds	Tester-driven iterations: generated map, ore economy, build menu, research, development director	Ongoing (to 2026-10-31)
WP7	Learning assembly: Beta sampler, binned posterior, kernel estimator, threshold environment, calibration	Planned
WP8	Build profiler (five features)	Planned
WP9	Strategy pool, shared threat cost table, multiple entries	Planned
WP10	Constraint and heuristic layers, bucketizer, heuristic-only director	Planned
WP11	Reward, credit assignment, full director	Planned
WP12	Harness: policies, counterfactuals, offline ladder	Planned
WP13	Gates G1–G3	Planned
WP14	Director UX in Unity (announcements) and session setup	Planned

5.2 Schedule

Semester 1 has twelve weeks (W5–W16); week 5 began on 14 September 2026. Filled squares are done, open squares planned, diamonds are gates.

Table 11: Semester 1 schedule

Task	W5	W6	W7	W8	W9	W10	W11	W12	W13	W14	W15	W16
Deterministic core, replay (WP0–WP5, WP-H)	■	■	■									
Combat, classes, shop, telemetry (WP-C1–C3)			■	■								
Tower UI, roster, balance check (WP-C4–C5)					■							
Gameplay rounds with testers					■	■	■					
Learning assembly (WP7)							□	□				
Build profile, strategies (WP8–WP9)								□	□			
Layers, director UX (WP10, WP14)									□			
Full director (WP11)										□		
Harness and ladder (WP12)										□	□	
Gates G1–G3 (WP13)											◆	
Report and session preparation												□

Table 12: Break and semester 2 schedule

Task	B1	B2	B3–4	W1–3	W4	W5	W6	W7–11	W12–13	W14–15
Pilot n = 5, gate G4	◆									
Threshold freeze (irreversible)		◆								
Buffer			□							
UX pass (route preview, announcements)				□						
Balance pass					□					
Second pilot, questionnaire reliability						□				
Build freeze (irreversible)							◆			
Main study, n ≈ 24 (G5–G7)								◆		
Analysis									□	
Writing and artifact release										□

If time runs short, scope is cut in a fixed order: (1) generated strategies (stretch goal), (2) the transfer condition, (3) the second pilot, (4) the sample from 24 to 18. Determinism, the balance pass and the build freeze are never cut.

5.3 Monitoring and Evaluation

Table 13: Input, process, output and benefit

Input	Process	Output	Benefit
Player commands (build, sell, upgrade, start) from mouse and keyboard; the map seed; the study condition.	The deterministic simulation runs each wave; at each wave boundary the director profiles the build, vetoes, scores and corrects, plans the next wave and updates its posteriors.	A wave shaped against the current build, announced before it starts; a replay and one telemetry row per wave; the session's placement entropy.	Players keep making structural decisions instead of repeating one layout; the thesis gets a reproducible, fair comparison of adaptive and static waves.

Progress is monitored per work package: each package has written acceptance criteria, is reviewed against them, and ends with an entry in the development log. Every experiment is written up as a result file that states the hypothesis before the table, the date, the commit and the exact command that reproduces it, including results that failed. Deviations from the plan are recorded as they happen and become the limitations chapter.

5.4 Risks

Table 14: Main risks and mitigation

Risk	Likelihood	Mitigation
Gameplay iteration delays the director work	High	Fixed checkpoint; the learning assembly depends only on the core and can run in parallel; cut order in 5.2
The learner shows no advantage over the heuristic (offline ladder)	Medium	Detected in semester 1; the heuristic-only director is a complete condition; report the result
Too little data per bin (G4)	Medium	Kernel prior for thin cells; fall back to 3 bins
Equal budgets still feel unequal to players	Medium	Fairness gate G6; tune only the shared cost table
A rule change breaks determinism	Low	Pinned replays run by every test run in both runtimes; state hash per tick

6. References

[1] Unity Technologies. (2026). Unity 6 Manual. https://docs.unity3d.com/Manual/

[2] Project documentation: Game Design Document (GDD), ARCHITECTURE.md, WORKPLAN.md and DEVLOG.md, in the project repository.

[3] R. Hunicke. (2005). The case for dynamic difficulty adjustment in games. In Proc. ACM SIGCHI Int. Conf. on Advances in Computer Entertainment Technology (ACE ’05), 429–433.

[4] M. Booth. (2009). The AI Systems of Left 4 Dead. Keynote, Fifth Artificial Intelligence and Interactive Digital Entertainment Conference (AIIDE ’09).

[5] R. Sutoyo, D. Winata, K. Oliviani and D. M. Supriyadi. (2015). Dynamic Difficulty Adjustment in Tower Defence. Procedia Computer Science, 59, 435–444.

[6] P. Spronck, M. Ponsen, I. Sprinkhuizen-Kuyper and E. Postma. (2006). Adaptive game AI with dynamic scripting. Machine Learning, 63(3), 217–248.

[7] W. Al Enezi and C. Verbrugge. (2023). [Full title to be added.] In Proc. AAAI Conf. on Artificial Intelligence and Interactive Digital Entertainment (AIIDE).

[8] E. Emerson. (2013). Crowd Pathfinding and Steering Using Flow Field Tiles. In S. Rabin (Ed.), Game AI Pro. CRC Press.

[9] R. Bellman. (1958). On a routing problem. Quarterly of Applied Mathematics, 16(1), 87–90.

[10] M. E. O’Neill. (2014). PCG: A Family of Simple Fast Space-Efficient Statistically Good Algorithms for Random Number Generation. Tech. Rep. HMC-CS-2014-0905, Harvey Mudd College.

[11] G. Fowler, L. C. Noll, K.-P. Vo and D. Eastlake. The FNV Non-Cryptographic Hash Algorithm. IETF Internet-Draft.

[12] W. R. Thompson. (1933). On the likelihood that one unknown probability exceeds another in view of the evidence of two samples. Biometrika, 25(3/4), 285–294.

[13] O. Chapelle and L. Li. (2011). An empirical evaluation of Thompson sampling. In Advances in Neural Information Processing Systems 24.

[14] S. Agrawal and N. Goyal. (2013). Thompson sampling for contextual bandits with linear payoffs. In Proc. 30th Int. Conf. on Machine Learning (ICML).

[15] L. Li, W. Chu, J. Langford and R. E. Schapire. (2010). A contextual-bandit approach to personalized news article recommendation. In Proc. 19th Int. Conf. on World Wide Web (WWW), 661–670.

[16] G. Marsaglia and W. W. Tsang. (2000). A simple method for generating gamma variables. ACM Transactions on Mathematical Software, 26(3), 363–372.

[17] G. W. Brier. (1950). Verification of forecasts expressed in terms of probability. Monthly Weather Review, 78(1), 1–3.

[18] C. Guo, G. Pleiss, Y. Sun and K. Q. Weinberger. (2017). On calibration of modern neural networks. In Proc. 34th Int. Conf. on Machine Learning (ICML).

[19] E. A. Nadaraya. (1964). On estimating regression. Theory of Probability and Its Applications, 9(1), 141–142.

[20] G. S. Watson. (1964). Smooth regression analysis. Sankhyā: The Indian Journal of Statistics, Series A, 26(4), 359–372.

[21] A. Garivier and E. Moulines. (2011). On upper-confidence bound policies for switching bandit problems. In Algorithmic Learning Theory (ALT), 174–188.

[22] C. E. Shannon. (1948). A mathematical theory of communication. Bell System Technical Journal, 27(3), 379–423.

[23] F. Wilcoxon. (1945). Individual comparisons by ranking methods. Biometrics Bulletin, 1(6), 80–83.

[24] D. Lakens. (2013). Calculating and reporting effect sizes to facilitate cumulative science: a practical primer for t-tests and ANOVAs. Frontiers in Psychology, 4, 863.